CHAPTER 1

DESIGNING A MEETING SCHEDULER

1 Getting Ready: The Meeting Scheduler Problem 2
2 Requirements for the Meeting Scheduler 4
3 Use Case Diagram for the Meeting Scheduler 5
4 Class Diagram for the Meeting Scheduler 9
5 Sequence Diagram for the Meeting Scheduler 18
6 Activity Diagram for the Meeting Scheduler 20
7 Code for the Meeting Scheduler 21
8 Summary: Designing a Meeting Scheduler 28
9 Key Insights and Tips: Designing a Meeting Scheduler . 33

This chapter shows the design of a meeting scheduler system, detailing the requirements, UML diagrams, and implementation code for the complete design.

1 Getting Ready: The Meeting Scheduler Problem

1.1 Problem definition

A **meeting scheduler** software enables organizations to efficiently arrange and manage meetings for multiple participants, ensuring optimal use of meeting rooms and everyone's time. The system helps find a suitable time and location by checking participants' availability, room capacity, and preferences. Organizers and participants can book, update, or cancel meetings; add or remove attendees; and receive real-time notifications for all meeting changes. When participants respond to invites or remove themselves from a meeting, their calendars are automatically updated to reflect their participation status.
In this LLD interview case study, you'll focus on:

- Assigning available meeting rooms based on room capacity, schedule, and participant count.
- Determining optimal meeting times by analyzing the availability of all required attendees.
- Managing the full meeting life cycle: creation, updates, cancellations, and participant management.
- Sending notifications for all meeting-related events, and updating calendars automatically when meetings are accepted, declined, or modified.
- Tracking each participant's invite/response status in the meeting record, and handling changes such as removals, cancellations, and overlapping bookings.

Note: This system model applies to any organization or platform that requires collaborative meeting management, from startups to enterprise-level operations.

Figure 1.1: Schedule a meeting using a meeting scheduler

1.2 Expectations from the interviewee

It is important to narrow down the components you will include in your design of a meeting scheduler. The following section provides an overview of some of the main expectations the interviewer will want to hear you discuss in more detail during the interview.

Room assignment

A meeting scheduler assigns a meeting room to a scheduled meeting. Make sure to ask the following questions of the interviewer to figure out how this assignment works:

- How does the system determine available rooms?
- How important is the capacity of a room when assigning a room for a meeting?

Availability of attendees

Multiple attendees are in a meeting, and all attendees have different schedules. You may ask the following questions to understand how the scheduler works:

- How does the system check the availability of the attendees?
- How does the system access the meeting information of all attendees?

1.3 Design approach

We will design this meeting scheduler system using the bottom-up design approach. For this purpose, we will follow the steps below:

- First, we'll identify the simple core entities such as `MeetingRoom`, `Interval`, `Participant`, and `Calendar`, and define their responsibilities and relationships.
- Next, we'll model how meetings are created, how the scheduler checks room and participant availability, assigns suitable rooms, and sends notifications for bookings, cancellations, and updates.
- We'll ensure the design gracefully handles conflicts, last-minute updates, recurring meetings, and edge cases like overlapping events or room shortages.
- This approach will incorporate SOLID principles and design patterns, ensuring the system is scalable, maintainable, and adaptable to future needs.

Diagrams and sample code will illustrate the main workflows, class structures, and collaboration between entities.

1.4 Design pattern

During an interview, discussing the design patterns the meeting scheduler falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design. Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a meeting scheduler? Please elaborate on your choice(s).

Let's explore the requirements of the meeting scheduler in the next lesson.

2 Requirements for the Meeting Scheduler

This lesson outlines the functional and operational requirements for the Meeting Scheduler. Identifying and understanding these requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for Requirement and "n" is a natural number.

2.1 Requirement collection

The requirements for the meeting scheduler design problem are defined below:

- **R1:** The system must support a configurable number of meeting rooms, allowing organizations to add or manage rooms as needed.
- **R2:** Each meeting room must have a defined capacity, ensuring only meetings within the room's limits can be scheduled.
- **R3:** Meeting rooms can be booked for specific time intervals, provided they are available and not already reserved for overlapping times.

- **R4:** When a meeting is scheduled, updated, or canceled, notifications must be sent promptly to all invited participants.
- **R5:** Invited participants receive meeting invites regardless of their current availability. The system must track each participant's invitation response status (accepted, declined, pending) within the meeting record.
- **R6:** Every user must have access to a personal calendar to view, schedule, or cancel meetings, and to track all meeting invitations and responses.
- **R7:** The organizer must be able to add or remove participants after a meeting has been scheduled, and appropriate notifications must be sent for any changes.

- **R8:** The system must handle booking conflicts and overlapping meetings, providing clear feedback if a room or participant is booked for the requested time.

- **R9:** The organizer should be able to edit meeting details (such as title, agenda, room, time, and participant list), and all updates should trigger notifications.
- **R10:** The system should support meeting cancellations by the organizer or authorized participants and notify all invitees of the cancellation.
- **R11:** If a participant declines an invite or withdraws from a meeting, the meeting must be removed from their calendar, and their response status updated in the meeting record. The meeting should remain active for other participants unless canceled by the organizer.
- **R12:** Any updates to a meeting (e.g., time, room, or participants) should automatically update all relevant calendars and trigger notifications.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases for the meeting scheduler.

3 Use Case Diagram for the Meeting Scheduler

Let's build the use case diagram for the meeting scheduler and understand the relationships between its main actors and core functions. First , we'll define the key elements of our system, followed by the full set of use cases and their relationships.

3.1 System

Our system is the meeting scheduler, responsible for organizing meetings by managing rooms, times, invitations, participant responses, and calendar updates.

3.2 Actors

Now, we'll define the main actors of the meeting scheduler.

Primary actors

- **Organizer:** Schedules new meetings, cancels existing ones, books and releases meeting rooms, and manages participant lists.
- **Participant:** Receives meeting invitations, accepts or declines invites, and manages their calendar by attending or withdrawing from meetings.

3.3 Use cases

In this section, we'll define the use cases for the meeting scheduler. We have listed the use cases according to their interactions with a particular actor.

Organizer

- **Schedule meeting:** The organizer can create a new meeting by specifying the date, time interval, participants, agenda, and booking a suitable meeting room.
- **Cancel meeting:** This option allows the organizer to cancel a previously scheduled meeting, freeing the reserved meeting room and notifying all invited participants.
- **Add participant to meeting:** Lets the organizer add new participants to an already scheduled meeting, triggers invitation notifications and updating participant calendars.
- **Remove participant from meeting:** Permits the organizer to remove participants from a scheduled meeting, ensure that the removed users receive notifications and their calendars are updated.

Participant

- **Accept invitation:** A participant can accept a meeting invitation, which updates their calendar and changes their response status in the meeting record.
- **Reject invitation:** This option allows a participant to decline a meeting invitation, which removes the meeting from their calendar and updates their response status in the meeting record.
- **Attend meeting:** Represents the participant's attendance at a scheduled meeting.
- **Withdraw from meeting:** This option allows a participant to withdraw from a meeting they had previously accepted, updating their calendar and notifying the organizer.

Meeting scheduler

- **Send invite notification:** Sends notifications to participants when invited to a new or updated meeting.
- **Send cancellation notification:** Sends notifications to all participants when a meeting is canceled.

- **Update calendar:** Automatically updates users' calendars whenever a meeting is created, updated, canceled, or when participant statuses change.
- **Remove meeting from calendar:** A meeting entry is removed from a participant's calendar when they decline or withdraw from a meeting or when the meeting is canceled.
- **Book room:** Reserves a meeting room for a scheduled meeting.
- **Check room availability:** Verify if a meeting room is available and suitable for the desired time and participant count.
- **Release room:** Frees a reserved meeting room when a meeting is canceled.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Organizer	**Participant**	**Meeting Scheduler**
Schedule meeting	Accept invitation	Send invite notification
Cancel meeting	Reject invitation	Send cancelation notification
Add participant to meeting	Attend meeting	Update calendar
Remove participant from meeting	Withdraw from meeting	Remove meeting from calendar
3=	3=	Book room
		Check room availability
		Release room

Include

When an organizer schedules a meeting, they must first reserve a room, select participants, and set the meeting details. The system then sends invitations and updates the calendars of all invited participants. These actions are best represented using "include" relationships in the use case diagram:

- Schedule meeting includes:
  - **Check room availability:** The system checks whether a room is available before booking a room.
  - **Book room:** After confirming room availability, the system books a room for a meeting.

  - **Add participant to meeting:** The system adds participants to the meeting once the room is confirmed.
  - **Send invite notification:** Invite notifications are sent to all participants after the meeting is scheduled.
  - **Update calendar:** The meeting has been added to each participant's calendar to reserve the time slot.

When an organizer cancels a meeting, the system ensures the meeting room is released, all participants are notified, and the meeting is removed from their calendars:

- Cancel meeting includes:
  - **Release room:** The reserved room is made available for others once the meeting is canceled.
  - **Send cancellation notification:** All participants are notified of the meeting cancellation.
  - **Remove meeting from calendar:** The meeting is removed from the calendars of all invited participants.

When an organizer adds a new participant to a meeting, the system must invite the new participant and update their calendar:

- Add participant to meeting includes:
  - **Send invite notification:** The newly added participant is notified about their invitation.
  - **Update calendar:** The meeting has been added to the new participant's calendar.

When an organizer removes a participant from a meeting, the system must update the participant:

- Remove participant from meeting includes:
  - **Send cancellation notification:** The removed participant is notified that they are no longer part of the meeting.
  - **Remove meeting from calendar:** The meeting is removed from the participant's calendar.

When a participant accepts a meeting invitation, the meeting is added to their calendar:

- Accept invitation includes:
  - **Update calendar:** The meeting appears in the participant's calendar.

When a participant rejects a meeting invitation, the meeting is removed from their calendar:

- Reject invitation includes:
  - **Remove meeting from calendar:** The meeting is removed from the participant's calendar if it was previously there.

3.5 Use case diagram

Here's the use case diagram of the meeting scheduler:

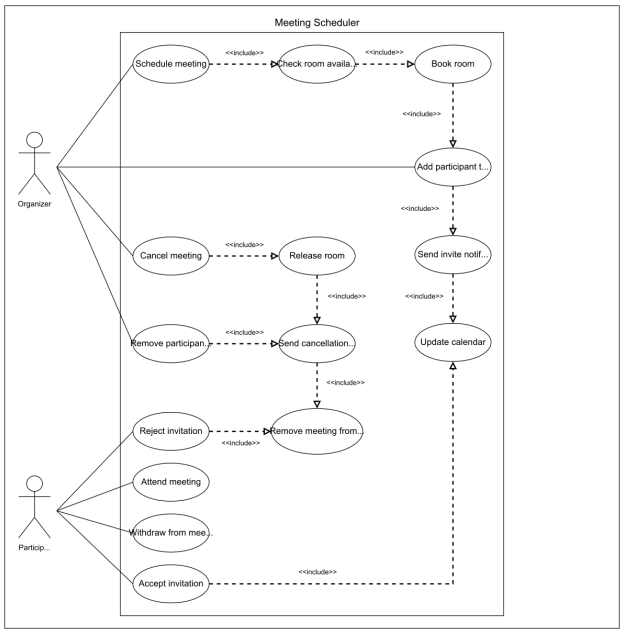


Figure 1.2: The use case diagram of the meeting scheduler

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Meeting Scheduler

Following a bottom-up approach in this lesson, we identify and design the main classes, abstract classes, and interfaces for the meeting scheduler system. Each class is tied to the requirements and use cases defined earlier, ensuring completeness, traceability, and real-world clarity.

4.1 Components of a meeting scheduler

As mentioned, we'll design the meeting scheduler using a bottom-up approach.

User

The `User` class represents a person who can organize or participate in meetings. It stores personal information (such as name and email), manages their calendar, and can respond to meeting invitations.
The class definition is shown below:

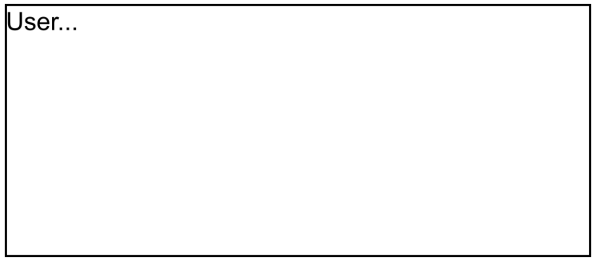


Figure 1.3: The class diagram of the User class

R4: When a meeting is scheduled, updated, or canceled, notifications must be sent promptly to all invited participants.
R5: Invited participants receive meeting invites regardless of availability. The system must track each participant's invitation response status (accepted, declined, pending) within the meeting record.
R6: Every user must have access to a personal calendar to view, schedule, or cancel meetings, and to track all meeting invitations and responses.

Interval

The `Interval` class contains the start time and end time of a meeting. The visual representation of the `Interval` class is as follows:

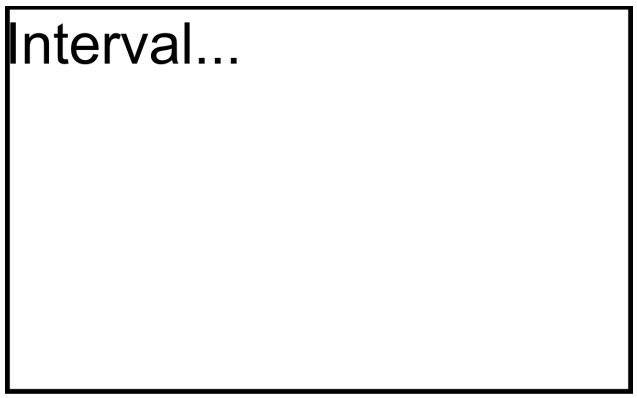


Figure 1.4: The class diagram of the Interval class

R3: Meeting rooms can be booked for specific time intervals, provided they are available and not already reserved for overlapping times.

MeetingRoom

The `MeetingRoom` class contains the details of any particular room, such as its capacity and status, to identify whether it is currently available. It also contains a list of intervals to track when the room is booked for a meeting.
The class diagram of the `MeetingRoom` class is provided below:

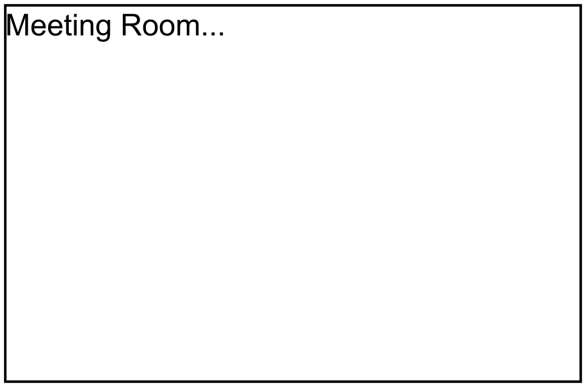


Figure 1.5: The class diagram of the MeetingRoom class

R1: The system must support a configurable number of meeting rooms, allowing organizations to add or manage rooms as needed.
R2: Each meeting room must have a defined capacity, ensuring only meetings within the room's limits can be scheduled.
R3: Meeting rooms can be booked for specific time intervals, provided they are available and not already reserved for overlapping times.
R8: The system must handle booking conflicts and overlapping meetings, providing clear feedback if a room or participant is already booked for the requested time.

Meeting

The `Meeting` class holds all details about a meeting, including the participants, their RSVP statuses, scheduled time, room, and subject.
The class diagram of the `Meeting` class is provided below:

R5: Invited participants receive meeting invites regardless of their current availability. The system must track each participant's invitation response status (accepted, declined, pending) within the meeting record.
R7: The organizer must be able to add or remove participants after a meeting has been scheduled, and appropriate notifications must be sent for any changes.
R9: Meeting details (such as title, agenda, room, time, and participant list) should be editable by the organizer, with all updates triggering notifications.
R10: The system should support meeting cancellations by the organizer or authorized participants and notify all invitees of the cancellation.

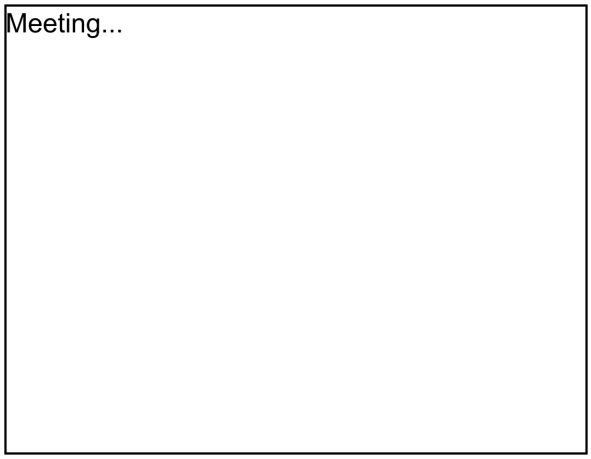


Figure 1.6: The class diagram of the Meeting class

Calendar

The `Calendar` class tracks all user meetings and allows adding or removing meetings based on user actions or meeting status changes.
The class definition is provided below:

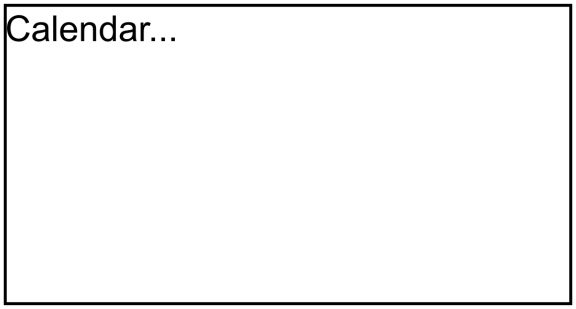


Figure 1.7: The class diagram of the Calendar class

R6: Every user must have access to a personal calendar to view, schedule, or cancel meetings, and to track all meeting invitations and responses.
R11: If a participant declines an invite or withdraws from a meeting, the meeting must be removed from their calendar, and their response status updated in the meeting record. The meeting should remain active for other participants unless canceled by the organizer.

MeetingScheduler

The `MeetingScheduler` class coordinates the overall scheduling process. It manages the list of meeting rooms, schedules and cancels meetings, checks room availability, books and releases rooms, and triggers notifications.
The visual representation of the `MeetingScheduler` class is provided below:

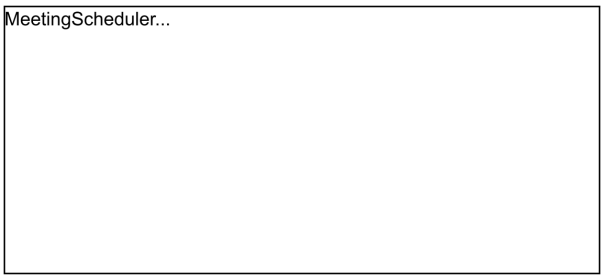


Figure 1.8: Class diagram of the MeetingScheduler class

R1: The system must support a configurable number of meeting rooms, allowing organizations to add or manage rooms as needed.
R2: Each meeting room must have a defined capacity, ensuring only meetings within the room's limits can be scheduled.
R3: Meeting rooms can be booked for specific time intervals, provided they are available and not already reserved for overlapping times.
R7: The organizer must be able to add or remove participants after a meeting has been scheduled, and appropriate notifications must be sent for any changes.
R8: The system must handle booking conflicts and overlapping meetings, providing clear feedback if a room or participant is already booked for the requested time.
R10: The system should support meeting cancellations by the organizer or authorized participants and notify all invitees of the cancellation.
R12: Any updates to a meeting (e.g., time, room, or participants) should automatically update all relevant calendars and trigger notifications.

Notification

The `Notification` class will send a notification for an invitation to a user regarding any new meeting. It will also send a cancellation notification to a user in case any meeting gets canceled or is postponed.
The UML representation of the class is shown below:

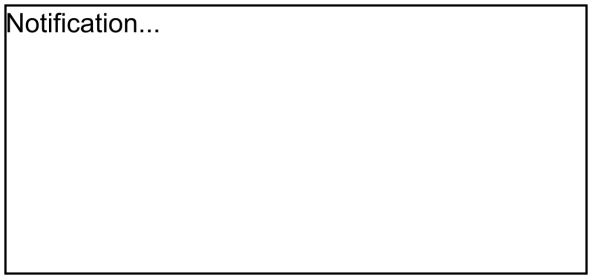
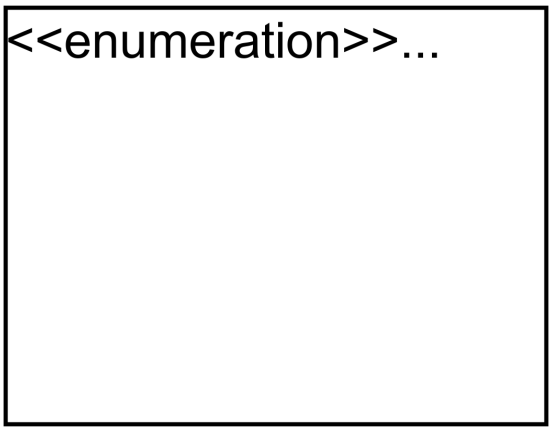


Figure 1.9: The class diagram of the Notification class

R4: A notification should be sent to everyone invited to the meeting.

Enumerations

Enumeration is generally a data type in which only a specific set of constants can be stored. Following is the list of enumerations required in the meeting scheduler system:
RSVPStatus (Enum): Tracks the current status of each participant's response to a meeting invite.



R5: Invited participants receive meeting invites regardless of their current availability. The system must track each participant's invitation response status (accepted, declined, pending) within the meeting record.

4.2 Relationship between the classes

Now, we'll discuss the relationships between the above-mentioned classes in the meeting scheduler.

Association

The class diagram has the following association relationships:

- `User` and `Meeting`:
  - Each `User` can be an organizer or a participant in multiple `Meeting` instances. Each `Meeting` maintains a mapping (Map<User, RSVPStatus>) of all its participants and their current invitation status.
- `Meeting` and `MeetingRoom`:
  - Each `Meeting` is assigned to a single `MeetingRoom`. Each `MeetingRoom` can host many meetings (at different intervals).
- `Notification` and `User`:

- `Notification` objects are sent to users whenever a meeting is scheduled, updated, or canceled.

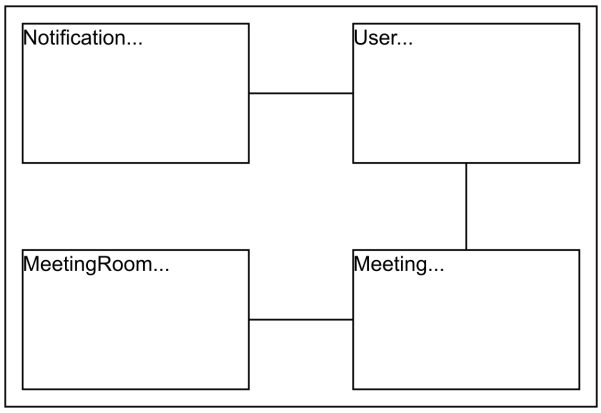


Figure 1.10: The association relationship between classes

Composition

The class diagram has the following composition relationships:

- `User` and `Calendar`:
  - Each `User` composes exactly one `Calendar`. If the user is deleted, their calendar is deleted as well.
- `Calendar` and `Meeting`:
  - Each `Calendar` comprises multiple `Meeting` objects (as the user accepts or schedules meetings).
- `Meeting` and `Interval`:
  - Each `Meeting` is scheduled for a single `Interval`. Each `MeetingRoom` keeps a list of all booked `Intervals`.

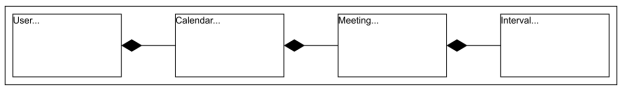


Figure 1.11: The composition relationship between classes

Aggregation

The class diagram has the following aggregation relationships:

- `MeetingScheduler` and `MeetingRoom`:
  - The `MeetingScheduler` aggregates all available `MeetingRoom` instances and manages their booking and release.
- `MeetingScheduler` and `Meeting`:
  - The `MeetingScheduler` aggregates and manages all scheduled `Meeting` instances for the organization.
- `MeetingRoom` and `Interval`:
  - Each `MeetingRoom` aggregates a collection of booked `Interval` s, representing its occupied time slots.

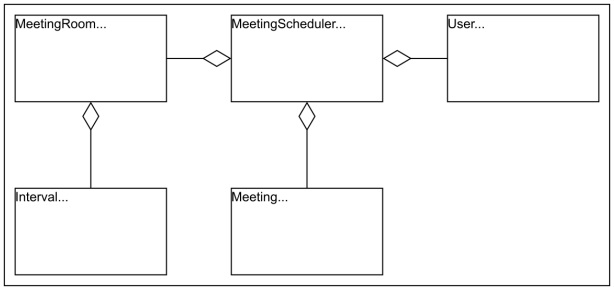


Figure 1.12: The aggregation relationship between classes

4.3 Class diagram of the meeting scheduler

This section outlines the multiplicity (cardinality) relationships between the main classes in our meeting scheduler. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	Target	Multiplicity	Reason
User	Calendar	1–1	Each user has exactly one calendar, and each calendar belongs to one user.
User	Meeting	1–0..*	A user can organize or participate in multiple meetings (including zero).
MeetingRoom	Meeting	1–0..*	A room can host zero or more meetings over time.
Meeting	Interval	1–1	Each meeting occupies exactly one time interval (start and end).
MeetingRoom	Interval	1–0..*	Each room tracks all intervals it is booked for.
Calendar	Meeting	1–0..*	A user's calendar contains all meetings associated with that user.
MeetingScheduler	MeetingRoom	1–1..*	The scheduler manages one or more meeting rooms.
MeetingScheduler	Meeting	1–1..*	The scheduler manages zero or more meetings.
User	MeetingScheduler	1–0..*	The scheduler can manage one or more users.

Here's the complete class diagram for the meeting scheduler:

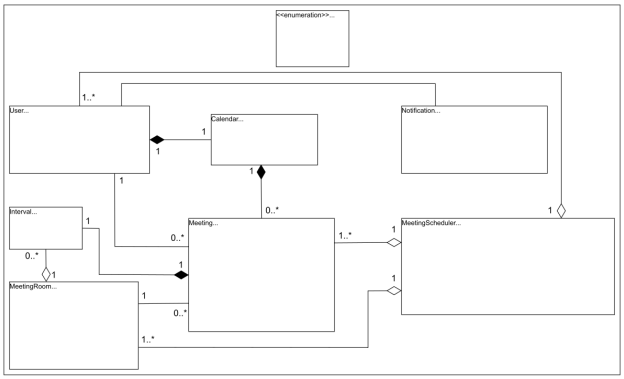


Figure 1.13: The class diagram of the meeting scheduler

4.4 Design pattern

Several classic object-oriented design patterns are applied in the meeting scheduler system to ensure a robust, scalable, and maintainable design.

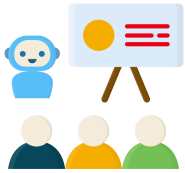
- **Singleton pattern:** The core scheduling functionality is centralized in the `MeetingScheduler` class, ensuring that only one active instance is responsible for managing all meetings,

rooms, and scheduling operations across the system. This is achieved using the Singleton pattern, which restricts the instantiation of the scheduler to a single, globally accessible object.

- **Observer pattern:** For user notifications, the system employs the Observer pattern. When a meeting is scheduled, updated, or canceled, all relevant users (participants or organizers) are notified automatically. This decouples the notification logic from the rest of the business workflow and ensures users always receive timely updates.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to assist you in strengthening your understanding of the concepts.



We have completed the class diagram of the meeting scheduler according to the requirements. In the next lesson, let's design the sequence diagram of the meeting scheduler.

5 Sequence Diagram for the Meeting Scheduler

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be multiple sequence diagrams for our meeting scheduler; here, we focus on the flow for scheduling a new meeting.
In this lesson, we will create sequence diagrams for the following interactions:

- **Schedule a meeting:** The meeting organizer schedules a meeting time for some attendees.
- **Sequence challenge:** The meeting organizer cancels a scheduled meeting.

5.1 Schedule a meeting

The sequence diagram to schedule a meeting should have the following actors and objects that will interact with each other:

- **Actor:** `Organizer`
- **Objects:** `MeetingScheduler`, `MeetingRoom`, `Meeting`, and `Calendar`

The steps in the schedule meeting interaction are listed below:

1. The organizer requests that a meeting be scheduled, specifying the list of participants, the time interval, and the meeting subject.
2. The `MeetingScheduler` checks for the availability of a suitable `MeetingRoom` for the given interval and required capacity.
3. If a room is available:
   (a) The `MeetingScheduler` books the room for the requested interval.
   (b) The `MeetingScheduler` creates a new `Meeting` and assigns the organizer and all participants. By default, all participant statuses are set to "PENDING."
   (c) The `MeetingScheduler` adds the new meeting to each participant's calendar (including the organizer).
   (d) The `MeetingScheduler` sends an invitation notification to each participant.
   (e) The `MeetingScheduler` informs the organizer that the meeting has been scheduled successfully.
4. Else, if no room is available:
   (a) The `MeetingScheduler` informs the organizer to select another meeting time or room.

Based on the order above, the sequence diagram for scheduling a meeting in the meeting scheduler system is given below:

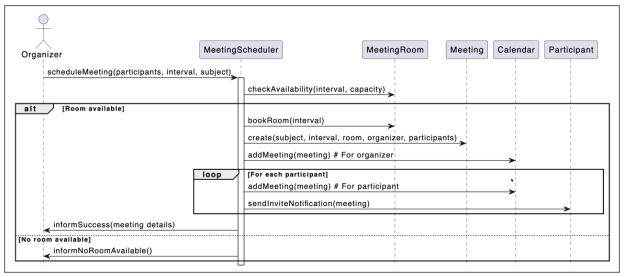


Figure 1.14: The sequence diagram for scheduling a meeting

5.2 Sequence challenge: Cancel meeting

You'll help us to complete a sequence diagram for the cancel meeting interaction. A skeleton of the cancel meeting sequence diagram is provided below:

Notice that the arrows in the diagram above are numbered from 1 to 7. The message boxes shown below are the messages to be exchanged between the actor(s) and object(s). Can you rearrange the messages below in the correct sequence in order they should appear in the skeleton of the sequence diagram given above?

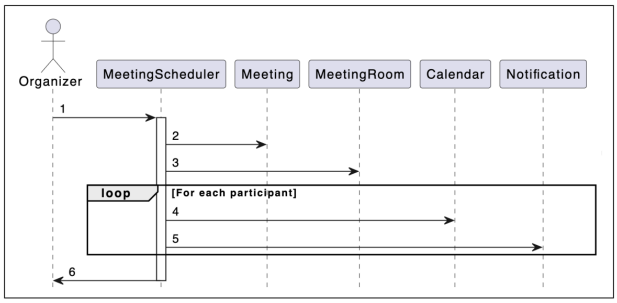


Figure 1.15: The sequence diagram for canceling a meeting

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete sequence diagram for the cancel meeting interaction.

Next, let's look at the activity diagrams for the meeting scheduler to understand the control flow of the system.

6 Activity Diagram for the Meeting Scheduler

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our meeting scheduler. In this lesson, we will create activity diagrams for the following two activities:

- Schedule a meeting.
- **Activity challenge:** Respond to an invite.

6.1 Schedule meeting

The following states and actions will be involved in this activity diagram.

States

Initial state: The user opens the calendar.
Final state: The notification and meeting details are sent to all the invited participants.

Actions

The user opens the calendar and selects an available slot. The user books a meeting room, and the meeting details are sent to all invited users.

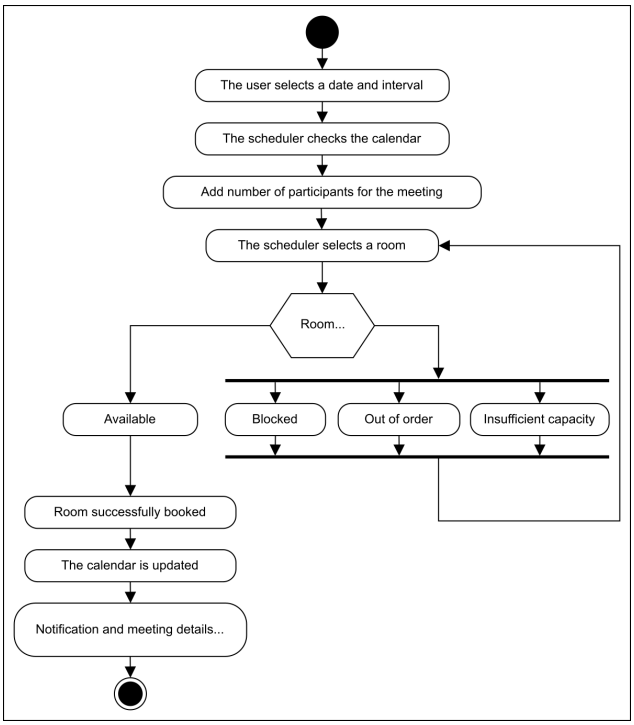


Figure 1.16: The activity diagram for a user scheduling a meeting

6.2 Activity challenge: Respond to an invite

You'll help us create an activity diagram of a person responding to an invite.
A skeleton of the activity diagram is given below:

Notice that the actions in the diagram above are numbered from 1 to 6. The slots below represent the activities, and the arrows represent the flow from one activity to another.
Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of our meeting scheduler's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Meeting Scheduler

We've reviewed the different aspects of the meeting scheduler and observed the attributes attached to the problem using various UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the meeting scheduler using multiple languages. This is usually the last step in an object-oriented design interview process.

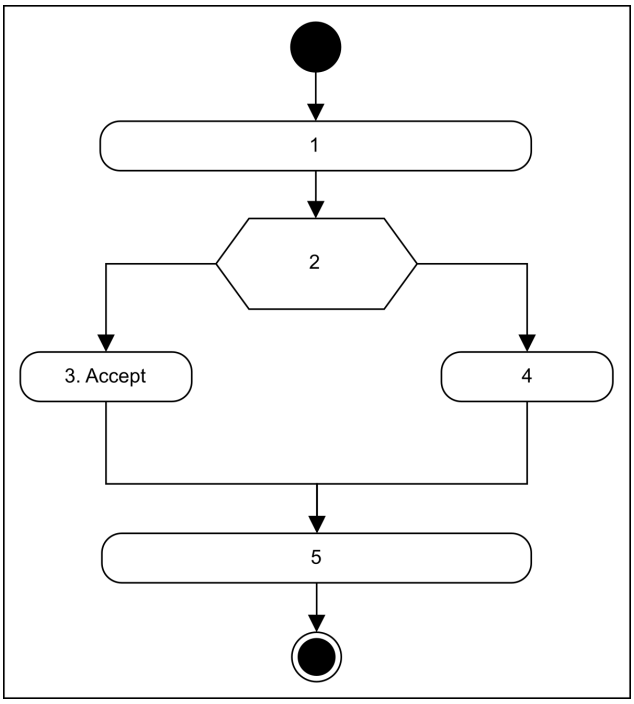


Figure 1.17: The activity diagram for a user responding to an invite

We have chosen the following languages to write the skeleton code of the different classes present in the meeting scheduler:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Meeting Scheduler.

7.1 Meeting scheduler classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumeration

The following code defines the enumeration and custom data type used in the meeting scheduler system.
`RSVPStatus`: We need to create an enumeration to keep track of the suit of states of the meetings, i.e., accepted, rejected, or pending.

The RSVPStatus enum

```
public enum RSVPStatus {
    ACCEPTED,
    REJECTED,
    PENDING
}
```

User

The `User` class refers to a participant taking part in a meeting. A user can either accept or reject an invitation. The definition of this class is given below:

The User class

```
public class User {
    private String name;
    private String email;
    private Calendar calendar;

    public User(String name, String email) { /* ... */ }

    public void respondInvitation(Meeting meeting, RSVPStatus response) { /* ... */
        }
    public List<Meeting> viewMeetings() { /* ... */ }

    // Getters and setters...
}
```

Interval

The `Interval` class denotes the meeting interval (the start and end time).

The Interval class

```
import java.util.Date;

public class Interval {
    private Date startTime;
    private Date endTime;
```

```
    public Interval(Date startTime, Date endTime) { /* ... */ }

    public Date getStartTime() { /* ... */ }
    public Date getEndTime() { /* ... */ }
}
```

MeetingRoom

The `MeetingRoom` class will represent the meeting rooms, each having a specific capacity, a boolean to check if a room is available, and a list of intervals for which the room is booked. The definition of the class is provided below:

The MeetingRoom class

```
import java.util.List;

public class MeetingRoom {
    private int id;
    private int capacity;
    private List<Interval> bookedIntervals;
    private boolean isAvailable;

    public MeetingRoom(int id, int capacity) { /* ... */ }

    public boolean isAvailableFor(Interval interval, int capacity) { /* ... */ }
    public void bookInterval(Interval interval) { /* ... */ }
    public void releaseInterval(Interval interval) { /* ... */ }

    // Getters and setters...
}
```

Calendar

The `Calendar` class contains a list of meetings to keep track of all the scheduled meetings. The definition of this class is provided below:

The Calendar class

```
import java.util.List;

public class Calendar {
    private List<Meeting> meetings;

    public Calendar() { /* ... */ }

    public void addMeeting(Meeting meeting) { /* ... */ }
    public void removeMeeting(Meeting meeting) { /* ... */ }
    public List<Meeting> getMeetings() { /* ... */ }
}
```

Meeting

The `Meeting` class outlines the meeting details, such as the number and list of participants, the meeting time interval, and the meeting room. It also has the option to add more participants. The definition of this class is shown below:

The Meeting class

```
import java.util.List;
import java.util.Map;

public class Meeting {
    private int id;
    private Map<User, RSVPStatus> participantStatus;
    private Interval interval;
    private MeetingRoom room;
    private String subject;

    public Meeting(int id, List<User> participants, Interval interval, MeetingRoom
        room, String subject) { /* ... */ }

    public void addParticipants(List<User> participants) { /* ... */ }
    public void updateParticipantStatus(User user, RSVPStatus status) { /* ... */ }
    public List<User> getAcceptedParticipants() { /* ... */ }
    public List<User> getPendingParticipants() { /* ... */ }
    public List<User> getRejectedParticipants() { /* ... */ }

    // Getters and setters...
}
```

MeetingScheduler

The `MeetingScheduler` class is the main class of the meeting scheduler and contains the organizer responsible for scheduling and canceling a meeting and booking or releasing a room. It also checks if any meeting rooms are available. In addition, there will be only one instance of the scheduler in the meeting scheduler. Therefore, the `MeetingScheduler` class will be a Singleton class to ensure that only one instance of the scheduler is created in the entire system.

The definition of this class is shown below:

The MeetingScheduler class

```
import java.util.List;

public class MeetingScheduler {
```

```
    private User organizer;
    private Calendar calendar;
    private List<MeetingRoom> rooms;

    public MeetingScheduler(User organizer, List<MeetingRoom> rooms) { /* ... */ }

    public Meeting scheduleMeeting(List<User> users, Interval interval, String
        subject) { /* ... */ }
    public boolean cancelMeeting(Meeting meeting) { /* ... */ }
    public MeetingRoom checkRoomsAvailability(int capacity, Interval interval) { /*
         ... */ }
    public boolean bookRoom(MeetingRoom room, Interval interval) { /* ... */ }
    public boolean releaseRoom(MeetingRoom room, Interval interval) { /* ... */ }

    // Getters and setters...
}
```

Notification

The `Notification` class is responsible for sending notifications to users about any new meetings cancellations. The definition of this class is provided below:

The Notification class

```
import java.util.Date;

public class Notification {
    private int notificationId;
    private String content;
    private Date creationDate;

    public Notification(int notificationId, String content, Date creationDate) { /*
         ... */ }

    public void sendInvite(User user, Meeting meeting) { /* ... */ }
    public void sendCancelNotification(User user, Meeting meeting) { /* ... */ }
}
```

7.2 Executable code: Meeting scheduler

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Meeting scheduler. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Sets up meeting rooms, users, and the meeting scheduler.

- **Scenario 1 (Schedule a meeting):** The organizer schedules a meeting with participants, the system checks room availability, books the room, adds the meeting to user calendars, and sends invitation notifications. Each participant randomly accepts or rejects the invite.
- **Scenario 2 (Schedule another meeting):** This scenario demonstrates scheduling another meeting using the same invitation and response workflow.
- **Scenario 3 (Cancel a meeting):** The organizer cancels the meeting, which releases the room, removes the meeting from all calendars, and sends cancellation notifications to participants.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: Meeting scheduler

```
import java.util.Date;

public class Driver {
    public static void main(String[] args) {
        // Set up rooms
        MeetingRoom roomA = new MeetingRoom(1, 4);
        MeetingRoom roomB = new MeetingRoom(2, 8);
        java.util.List<MeetingRoom> rooms = java.util.Arrays.asList(roomA, roomB);

        // Set up users
        User alice = new User("Alice", "alice@email.com");
        User bob = new User("Bob", "bob@email.com");
        User charlie = new User("Charlie", "charlie@email.com");
        java.util.List<User> participants = java.util.Arrays.asList(alice, bob,
            charlie);

        // Set up scheduler
        MeetingScheduler scheduler = new MeetingScheduler(alice, rooms);

        // Scenario 1: All accept
        header("Scenario 1: Schedule a Meeting (Random Accept/Reject)");
        arrow("Scheduling meeting "Design Review" for Alice, Bob, Charlie...");
        Interval interval1 = getInterval(2025, java.util.Calendar.JULY, 10, 10, 0,
            11, 0);
        Meeting meeting1 = scheduler.scheduleMeeting(participants, interval1, "
            Design Review");

        // Scenario 2: Schedule another meeting (possibly some will reject)
        header("Scenario 2: Schedule Another Meeting (Random Accept/Reject)");
        arrow("Scheduling meeting "Sprint Planning" for Alice, Bob, Charlie...");
        Interval interval2 = getInterval(2025, java.util.Calendar.JULY, 10, 12, 0,
            13, 0);
        Meeting meeting2 = scheduler.scheduleMeeting(participants, interval2, "
            Sprint Planning");
```

```
        // Scenario 3: Cancel a meeting
        header("Scenario 3: Cancel Meeting");
        arrow("Cancelling meeting "Design Review"...");
        if (meeting1 != null) {
            scheduler.cancelMeeting(meeting1);
        }
    }

    // Formatting helpers
    private static void header(String title) {
        System.out.println("
==============================");
        System.out.println(" " + title);
        System.out.println("==============================
");
    }
    private static void arrow(String msg) { System.out.println(" " + msg); }

    // Utility function for creating an interval
    private static Interval getInterval(int year, int month, int day, int startHour
        , int startMin, int endHour, int endMin) {
        java.util.Calendar cal = java.util.Calendar.getInstance();
        cal.set(year, month, day, startHour, startMin, 0);
        Date start = cal.getTime();
        cal.set(year, month, day, endHour, endMin, 0);
        Date end = cal.getTime();
        return new Interval(start, end);
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of the meeting scheduler, including how it can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Meeting Scheduler

Now that you've completed the meeting scheduler case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

The following are the primary functional and operational requirements for the meeting scheduler system:

1. Support a configurable number of meeting rooms with editable capacities.
2. Prevent room overbooking by checking availability and overlapping intervals.
3. Automatically notify participants when meetings are scheduled, updated, or canceled.
4. Track each participant's invitation response (accepted, declined, pending).
5. Provide each user with a personal calendar for meeting management.
6. Allow organizers to add or remove participants and notify them accordingly.
7. Handle scheduling conflicts and overlapping meetings with clear feedback.
8. Allow editing of meeting details with real-time updates and notifications.
9. Allow organizers or authorized users to cancel meetings and inform attendees.
10. Remove a meeting from a participant's calendar if they decline or withdraw.
11. Autoupdate calendars and send notifications when any meeting property changes.

8.2 System actors

The system involves both primary and secondary actors who interact with or support the core functionalities:

- **Organizer:** Schedules, edits, or cancels meetings and manages the list of participants.
- **Participant:** Receives invitations, accepts/declines meetings, and maintains their calendar.

8.3 Important classes and relationships

The system comprises several core classes, each with defined responsibilities and relationships.

Class	Description	Important Relationships
User	Represents a person who organizes or participates in meetings.	Composes one Calendar; associated with multiple Meeting instances.
Interval	Defines a time slot using start and end times.	Aggregated in Meeting and MeetingRoom.
MeetingRoom	Represents a meeting room with capacity and booked intervals.	Aggregates Interval; associated with multiple Meeting instances.
Meeting	Contains meeting details: time, room, participants, and RSVP statuses.	Associated with User, MeetingRoom, and Interval; composed in Calendar.
Calendar	Maintains a list of a user's meetings.	Composed in User; composed of multiple Meeting instances.
MeetingScheduler	Central scheduler managing all meetings and rooms.	Aggregates Meeting and MeetingRoom; handles booking, releasing, notifications.
Notification	Sends invites and cancellations to users.	Associated with User.
RSVPStatus	Enum representing a participant's response status.	Used in Meeting for tracking participant responses.

8.4 Design highlights

This section summarizes the overall design approach and highlights how key principles and patterns were applied to develop the meeting scheduler system.

Design approach

The system was designed using a bottom-up approach, starting with foundational classes like `User`, `MeetingRoom`, `Interval`, and `Calendar`. These components were then composed and aggregated to model complex interactions such as scheduling, notifications, and conflict handling.

Design principles

The design of the meeting scheduler reflects key SOLID principles:

- **Single Responsibility principle (SRP):** Each class is responsible for only one specific task (e.g., `Notification` only sends messages).
- **Open/Closed principle (OCP):** The system supports extending functionality (e.g., more notification types) without modifying existing code.
- **Liskov Substitution principle (LSP):** All user types (organizers and participants) can be treated uniformly as `User` objects.

- **Interface Segregation principle (ISP):** Interfaces are implied through class responsibilities without forcing unnecessary dependencies.
- **Dependency Inversion principle (DIP):** High-level modules like `MeetingScheduler` rely on abstractions like `Notification`, decoupling system parts.

Design patterns

Several classical design patterns were applied to improve system maintainability:

- **Singleton pattern:** Ensures only one instance of `MeetingScheduler` manages the organization's meetings and rooms.
- **Observer pattern:** Used for sending notifications to users when meetings are created, updated, or canceled, decoupling the messaging system from scheduling logic.

8.5 Object interactions

The following list describes how key objects interact dynamically during different system events:

- **Schedule meeting:**
  - The organizer provides meeting details.
  - `MeetingScheduler` checks available `MeetingRoom` based on `Interval` and capacity.
  - If available, the room is booked and a `Meeting` is created.
  - `Meeting` is added to each participant's `Calendar`.
  - Notifications are sent via `Notification` to all participants.
- **Cancel meeting:**
  - The organizer cancels the meeting.
  - `MeetingScheduler` releases the `MeetingRoom`.
  - `Meeting` is removed from participants' `Calendars`.
  - Cancellation notifications are dispatched.
- **Respond to invite:**
  - Participant accepts or declines an invite.
  - Their `Calendar` is updated accordingly.
  - `Meeting` record is updated with their new `RSVPStatus`.

- **Update meeting:**
  - Organizer edits details (e.g., room, time).
  - `MeetingScheduler` checks for conflicts, rebooks if needed.
  - Calendars and notifications are updated.

8.6 System workflow

This section outlines the main workflows modeled in the system, derived from activity diagrams and use cases:

- **The organizer schedules a meeting:**
  - Opens calendar, selects time, sets agenda, and participants.
  - `MeetingScheduler` checks and books an available room.
  - Meeting is created, calendars updated, notifications sent.
- **Participant responds to an invitation:**
  - Opens calendar, views meeting invite.
  - Accepts or declines the meeting.
  - Calendar and RSVP status are updated.
- **The organizer adds a participant:**
  - A new participant is added to the meeting.
  - Their calendar is updated, and they receive an invitation.
- **The organizer removes a participant:**
  - Participant is removed from the meeting.
  - Calendar entry is deleted, and a cancellation notice is sent.
- **The organizer cancels a meeting:**
  - The meeting is removed from all calendars.
  - The room is released, and all participants are notified.

9 Key Insights and Tips: Designing a Meeting Scheduler

Congratulations on completing the case study for designing a meeting scheduler. This lesson builds on your progress by outlining common design mistakes, sharing focused interview tips, and offering a short quiz to test your understanding. You 'll also find related case studies to reinforce the object-oriented concepts introduced in this system.

9.1 Common mistakes

Avoiding these common mistakes will help you create a more reliable and scalable meeting scheduler:

- **Ignoring participant availability:** Failing to check if invitees are free at the proposed time can lead to poor scheduling. Ensure the scheduler verifies each participant's calendar before finalizing the meeting.
- **Room booking without time conflict checks:** Overlooking existing reservations may result in double-bookings. Validate available intervals in the `MeetingRoom` object before confirming a room.
- **Failing to model user responses:** Assuming all participants will attend by default can lead to incorrect scheduling. Track user responses explicitly in the `Meeting` object to accurately reflect accepted and declined invites.
- **Not modeling user responses:** Overlooking that users can reject invites and assuming universal attendance reduces accuracy. Track user responses and reflect them in the meeting object state.
- **Lack of a centralized notification mechanism:** Many candidates overlook how notifications are generated and sent when meetings are created, updated, or canceled. Introduce a dedicated `Notification` class or service to consistently manage email or in-app alerts across workflows.

9.2 Key interview tips

Understand key design patterns, system components, and real-world applications to prepare for your interview and confidently discuss your approach.

9.3 Test your understanding

Now that you understand OOD principles well, solve the quiz below to test your knowledge.

Tip	Explanation
Emphasize bottom-up design.	Mention starting with classes like `Interval` and `MeetingRoom`, then building up to `Scheduler`. This shows methodical, scalable thinking.
Discuss calendar and user modeling.	Show how the `User` has a `Calendar` object with available intervals and meetings, supporting availability checks and updates.
Explain the meeting life cycle clearly.	Walk through scheduling, invite response, and cancellation—including how data flows across `Meeting`, `Calendar`, and `Notification` classes.
Include failure and edge case handling.	Describe how you manage unavailable rooms, partial acceptances, or rejected invites. This demonstrates practical and robust thinking.
Tie class design to real-world flows.	Map each class (e.g., `Notification`, `Calendar`, `MeetingRoom`) to a real process users experience—interviewers love relatable design grounding.

Table 1.1: Tips for Interview Class Design Explanation

Designing a Meeting Scheduler

Question 1: Which class is most appropriate to store a user's upcoming meetings?

- `MeetingScheduler`
- `MeetingRoom`
- `User`
- **[CORRECT]** `Calendar`
  Explanation: The `Calendar` class tracks all scheduled meetings.

Question 2: Why should the `Meeting` object maintain a list of invitees and their responses?

- To reduce the size of the `Calendar` object.
- To allow users to change meeting times themselves.
- **[CORRECT]** To track attendance and handle accept/reject logic effectively
  Explanation: Maintaining invitees and their responses within the `Meeting` object allows the system to manage RSVP workflows, handle partial acceptances, and generate accurate notifications or updates.
- To prevent users from being invited to more than one meeting at once

Question 3: What would be a good way to avoid room double-booking?

- Add a flag to check if the room is used.
- Only allow booking in working hours.
- **[CORRECT]** Maintain a list of time intervals in `MeetingRoom` and validate before booking.
  Explanation: Validating against an interval list avoids time conflicts.
- Always ask the user before booking.

Question 4: Which class is primarily responsible for checking time slot conflicts when scheduling a meeting?

- `User`
- `Calendar`
- `MeetingRoom`
- **[CORRECT]** `MeetingScheduler`
  Explanation: The `MeetingScheduler` coordinates between `Calendar` and `MeetingRoom` to check for conflicts and finalize bookings, acting as the central orchestrator for scheduling logic.

Question 5: If the `MeetingScheduler` class handles booking, user availability, and notification sending entirely by itself, which design principle is violated?

- Open/Closed principle
- Interface Segregation principle
- **[CORRECT]** Single Responsibility principle
  Explanation: Having `MeetingScheduler` manage multiple distinct concerns violates the Single Responsibility Principle (SRP). Responsibilities like booking, availability checks, and notification delivery should be delegated to specialized classes.
- Dependency Inversion principle

Related case studies

The following case studies will help reinforce concepts used in the meeting scheduler system:

- **Designing a calendar app:** Explores a similar structure with user-specific events, reminders, and daily/weekly views. Like the Meeting Scheduler, it focuses on time-slot management and availability.
- **Designing a restaurant management system:** Manages table reservations, time slots, and guest preferences, often with cancellation and rebooking logic. Like a meeting scheduler, it requires handling concurrent bookings and real-time availability updates.
- **Designing a hotel management system:** Involves booking rooms based on time, capacity, and guest preferences. Reinforces concepts like calendar views, cancellations, and notifications, closely matching the meeting scheduler's workflows.
- **Designing Facebook:** While broader in scope, this system emphasizes real-time event notifications, user interactions, and calendar-driven scheduling features—skills that directly map to meeting invites and updates.
- **Designing an airline management system:** Handles booking, seat availability, time-bound resource allocation, and customer notifications. It parallels meeting scheduling by focusing on capacity, location, and timing coordination.